STAT 293: S&P 500 Data cleaning and analysis

Walter Navarro

2026-11-13

```
library(quantmod)
library(readr)
library(knitr)
```

Load the libraries

Identify stocks common to both years We use a publicly available reconstruction of past S&P 500 membership. We cross the membership snapshots in the beginning of 2020 and 2025. This is not to say that membership has to be maintained for both years

```
# Read the historical membership file
url <- "https://raw.githubusercontent.com/chinobing/historical_sp500_constituents/main/sp_500_historical

membership <- read_csv(url, col_types = cols(.default = col_character()), show_col_types = FALSE)

# Convert dates and put them in chronological order
membership$date <- as.Date(membership$date)
membership <- membership[!is.na(membership$date), ]
membership <- membership[order(membership$date), ]

# Get the most recent membership list on or before a date
get_members <- function(date) {
  date <- as.Date(date)
  rows <- which(membership$date <= date)

  if (length(rows) == 0) {
    stop("No membership snapshot available")
  }

  i <- tail(rows, 1)

  # Check that the snapshot is no more than 31 days old
  if (as.numeric(date - membership$date[i]) > 31) {
    stop("The available membership snapshot is too old")
  }

  cat("Snapshot used:", as.character(membership$date[i]), "\n")

  # Makes stock label into a vector
```

```
  trimws(strsplit(membership$tickers[i], ",", fixed = TRUE)[[1]])
}

# Get membership lists for the beginning of each year
members2020 <- get_members("2020-01-01")
```

```
## Snapshot used: 2019-12-23
```

```
members2025 <- get_members("2025-01-01")
```

```
## Snapshot used: 2025-01-01
```

```
# Keep stocks appearing in both lists
stocks <- sort(intersect(members2020, members2025))

# Replace dots with hyphens for Yahoo ticker symbols
stocks <- unique(gsub(".", "-", stocks, fixed = TRUE))

cat("Stock series:", length(stocks), "\n")
```

```
## Stock series: 407
```

Download prices and calculate returns We use the adjusted-close field from Yahoo. The observations of the previous December allow us to compute the initial return in each year. Excludes stocks with incomplete or incorrect histories

```
# Download adjusted prices for one stock and year
download_prices <- function(stock, year) {
  prices <- getSymbols(
    stock,
    src = "yahoo",
    from = paste0(year - 1, "-12-20"),
    to = paste0(year + 1, "-01-01"),
    auto.assign = FALSE
  )

  Ad(prices)
}

# Use SPY only to identify the reference trading dates
calendar2020 <- download_prices("SPY", 2020)
calendar2025 <- download_prices("SPY", 2025)

dates2020 <- as.Date(zoo::index(calendar2020))
dates2025 <- as.Date(zoo::index(calendar2025))

dates2020 <- dates2020[format(dates2020, "%Y") == "2020"]
dates2025 <- dates2025[format(dates2025, "%Y") == "2025"]

# Calculate returns
get_returns <- function(prices, dates) {
```

```
  values <- as.numeric(prices)

  if (any(!is.finite(values)) || any(values <= 0)) {
    stop("Invalid or missing adjusted prices")
  }

  returns <- diff(log(prices))
  position <- match(dates, as.Date(zoo::index(returns)))
  returns <- as.numeric(returns[position])

  if (any(!is.finite(returns))) {
    stop("Incomplete return history")
  }

  if (sd(returns) == 0) {
    stop("Zero return variance")
  }

  returns
}

# Store the returns
returns2020 <- list()
returns2025 <- list()

excluded <- data.frame(
  Stock = character(),
  Reason = character()
)

for (i in seq_along(stocks)) {
  stock <- stocks[i]

  result <- tryCatch({
    prices2020 <- download_prices(stock, 2020)
    prices2025 <- download_prices(stock, 2025)

    list(
      year2020 = get_returns(prices2020, dates2020),
      year2025 = get_returns(prices2025, dates2025)
    )
  }, error = function(e) {
    conditionMessage(e)
  })

  if (is.list(result)) {
    returns2020[[stock]] <- result$year2020
    returns2025[[stock]] <- result$year2025
  } else {
    excluded <- rbind(
      excluded,
      data.frame(Stock = stock, Reason = result)
    )
```

```
  }

  Sys.sleep(0.3)
}

if (length(returns2020) < 10) {
  stop("Too few stocks were downloaded successfully")
}

# Rows are trading days and columns are individual stocks
R2020 <- do.call(cbind, returns2020)
R2025 <- do.call(cbind, returns2025)

data2020 <- data.frame(
  date = dates2020,
  R2020,
  check.names = FALSE
)

data2025 <- data.frame(
  date = dates2025,
  R2025,
  check.names = FALSE
)

stopifnot(identical(colnames(R2020), colnames(R2025)))

kable(
  data.frame(
    Year = c(2020, 2025),
    Trading_days = c(nrow(R2020), nrow(R2025)),
    Stocks = c(ncol(R2020), ncol(R2025))
  ),
  caption = "Retained stock-return data"
)
```

Table 1: Retained stock-return data

Year	Trading_days	Stocks
2020	253	394
2025	250	394

```
kable(excluded, caption = "Excluded stock series")
```

Table 2: Excluded stock series

Stock	Reason
ANSS	Unable to import “ANSS”. cannot open the connection
AVB	Unable to import “AVB”.

Stock	Reason
cannot open the connection	
BK	Unable to import “BK”.
cannot open the connection	
DFS	Unable to import “DFS”.
cannot open the connection	
EA	Unable to import “EA”.
cannot open the connection	
EQR	Unable to import “EQR”.
cannot open the connection	
HES	Unable to import “HES”.
cannot open the connection	
HOLX	Unable to import “HOLX”.
cannot open the connection	
IPG	Unable to import “IPG”.
cannot open the connection	
JNPR	Unable to import “JNPR”.
cannot open the connection	
K	Unable to import “K”.
cannot open the connection	
MMC	Unable to import “MMC”.
cannot open the connection	
WBA	Unable to import “WBA”.
cannot open the connection	

Center and scale the returns We subtract the yearly mean for each stock and use the same pooled standard deviation throughout the two years

```
# Subtract each stock's average return within each year
C2020 <- scale(R2020, center = TRUE, scale = FALSE)
C2025 <- scale(R2025, center = TRUE, scale = FALSE)

# Calculate one pooled standard deviation for each stock
pooled_sd <- sqrt(
  (colSums(C2020^2) + colSums(C2025^2)) /
    (nrow(C2020) + nrow(C2025) - 2)
)

# Check that every standard deviation is finite and positive
stopifnot(
  all(is.finite(pooled_sd)),
  all(pooled_sd > 0)
)

# Divide each stock's centered returns by its pooled standard deviation
X2020 <- sweep(C2020, 2, pooled_sd, "/")
X2025 <- sweep(C2025, 2, pooled_sd, "/")
```

Principal component analysis PCA uses all available observations each year Differences in explained variance do not by se imply different loading spaces

```r
# Run PCA on the already centered and scaled returns
pca2020 <- prcomp(X2020, center = FALSE, scale. = FALSE)
pca2025 <- prcomp(X2025, center = FALSE, scale. = FALSE)

# Square the standard deviations to get component variances
variance2020 <- pca2020$sdev^2
variance2025 <- pca2025$sdev^2

# Divide by total variance to get the proportion explained
pve2020 <- variance2020 / sum(variance2020)
pve2025 <- variance2025 / sum(variance2025)

# Summarize the first ten components
# Cumulative percentages add the contributions up to each component
pca_table <- data.frame(
  PC = 1:10,
  Percent_2020 = 100 * pve2020[1:10],
  Percent_2025 = 100 * pve2025[1:10],
  Cumulative_2020 = 100 * cumsum(pve2020)[1:10],
  Cumulative_2025 = 100 * cumsum(pve2025)[1:10]
)

kable(pca_table, digits = 3)
```

PC	Percent_2020	Percent_2025	Cumulative_2020	Cumulative_2025
1	57.209	30.327	57.209	30.327
2	7.544	9.482	64.754	39.809
3	3.153	3.971	67.906	43.780
4	2.188	2.327	70.094	46.107
5	1.458	1.838	71.552	47.946
6	1.330	1.716	72.882	49.661
7	1.163	1.504	74.045	51.165
8	1.085	1.393	75.130	52.558
9	0.949	1.280	76.079	53.838
10	0.888	1.138	76.967	54.976

```r
# Summarize the first component and the first three together
presentation_table <- data.frame(
  Component = c(
    "First principal component",
    "First three principal components"
  ),
  Percent_2020 = 100 * c(pve2020[1], sum(pve2020[1:3])),
  Percent_2025 = 100 * c(pve2025[1], sum(pve2025[1:3]))
)

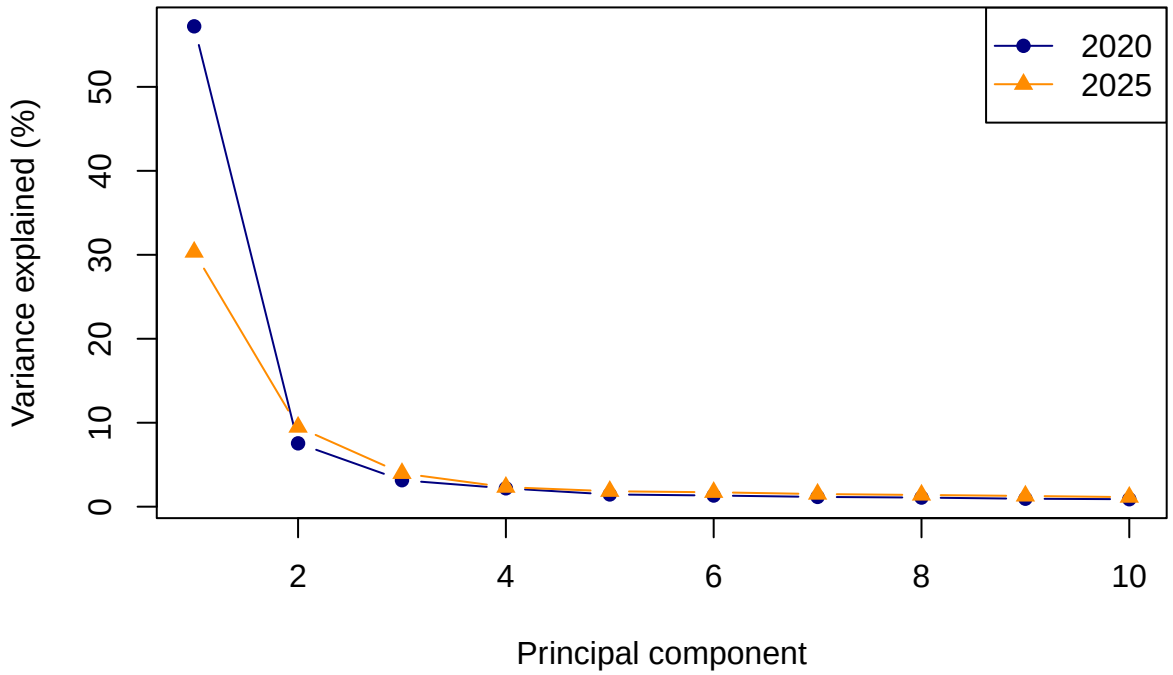
kable(presentation_table, digits = 3)
```

Component	Percent_2020	Percent_2025
First principal component	57.209	30.327

Component	Percent_2020	Percent_2025
First three principal components	67.906	43.780

```r
# Plot both years using the same axes
matplot(
  1:10,
  100 * cbind(pve2020[1:10], pve2025[1:10]),
  type = "b",
  pch = c(16, 17),
  lty = 1,
  col = c("navy", "darkorange"),
  xlab = "Principal component",
  ylab = "Variance explained (%)"
)

# Identify each year's line
legend(
  "topright",
  legend = c("2020", "2025"),
  col = c("navy", "darkorange"),
  pch = c(16, 17),
  lty = 1
)
```



Prepare equal sample sizes For the midpoint test, we save the first T observations in each year, where T is the maximum possible even sample size This corresponds to sample sizes, not calendar dates

```r
# Find the smaller number of observations across the two years
T <- min(nrow(X2020), nrow(X2025))
```

```r
# Round down to an even number for the midpoint split
T <- 2 * floor(T / 2)

# Keep the first T observations and subtract each column's mean
A <- scale(X2020[1:T, ], center = TRUE, scale = FALSE)
B <- scale(X2025[1:T, ], center = TRUE, scale = FALSE)

# Display the sample size and date range used for each year
sample_table <- data.frame(
  Year = c(2020, 2025),
  Test_observations = T,
  Stocks = ncol(A),
  First_date = c(dates2020[1], dates2025[1]),
  Last_date = c(dates2020[T], dates2025[T])
)

kable(sample_table)
```

Year	Test_observations	Stocks	First_date	Last_date
2020	250	394	2020-01-02	2020-12-28
2025	250	394	2025-01-02	2025-12-31

Estimate the loading matrices We provide a three-factor specification The loading entries themselves cannot be directly compared, since the factor representation is known only up to transformation

```r
# Choose three factors
r <- 3

# Calculate the PCA decompositions using SVD
pca_A <- svd(A, nu = r, nv = 0)
pca_B <- svd(B, nu = r, nv = 0)

# Keep the first three factor directions
U2020 <- pca_A$u[, 1:r, drop = FALSE]
U2025 <- pca_B$u[, 1:r, drop = FALSE]

# Normalize the factors so their sample second-moment matrix is the identity
F2020 <- sqrt(T) * U2020
F2025 <- sqrt(T) * U2025

# Estimate how each stock relates to the factors
Lambda2020 <- t(A) %*% F2020 / T
Lambda2025 <- t(B) %*% F2025 / T

# Display the estimated loadings for the first five stocks
loading_table <- data.frame(
  Stock = colnames(A)[1:5],
  Lambda2020[1:5, ],
  Lambda2025[1:5, ]
)
```

```r
# Label the columns by year and factor
names(loading_table) <- c(
  "Stock",
  "2020 Factor 1", "2020 Factor 2", "2020 Factor 3",
  "2025 Factor 1", "2025 Factor 2", "2025 Factor 3"
)

kable(loading_table, digits = 3)
```

	Stock	2020 Factor 1	2020 Factor 2	2020 Factor 3	2025 Factor 1	2025 Factor 2	2025 Factor 3
A	A	0.843	0.302	0.101	-0.696	-0.040	0.209
AAPL	AAPL	0.826	0.378	0.352	-0.554	-0.088	0.029
ABBV	ABBV	0.772	0.204	0.142	-0.389	0.303	0.045
ABT	ABT	0.838	0.611	0.021	-0.236	0.322	-0.089
ACN	ACN	0.989	0.165	0.189	-0.496	0.012	0.038

Define the transformation and Wald test This function corrects the factor count of original and transformed series to satisfy the previous presentation requirement. It does not use Bai-Ng factor selection

```r
common_structure_test <- function(A, B, r, bandwidth) {

  T <- nrow(B)

  # Check equal sample sizes and matching stock columns
  stopifnot(
    nrow(A) == T,
    T %% 2 == 0,
    identical(colnames(A), colnames(B)),
    bandwidth >= 1,
    bandwidth < T,
    bandwidth == floor(bandwidth)
  )

  # Calculate PCA decompositions for the original datasets
  pca_A <- svd(A, nu = 0, nv = r)
  pca_B <- svd(B, nu = 0, nv = r)

  # Keep orthonormal bases for the loading spaces
  Q1 <- pca_A$v[, 1:r, drop = FALSE]
  Q2 <- pca_B$v[, 1:r, drop = FALSE]

  # Split Dataset B into equal halves
  first <- 1:(T / 2)
  second <- (T / 2 + 1):T

  B_first <- B[first, , drop = FALSE]
  B_second <- B[second, , drop = FALSE]

  # Project each half onto its corresponding loading space
  projected_first <- B_first %*% Q1 %*% t(Q1)
```

```r
projected_second <- B_second %*% Q2 %*% t(Q2)

# Average the original and projected observations
Y <- B
Y[first, ] <- (B_first + projected_first) / 2
Y[second, ] <- (B_second + projected_second) / 2

# Estimate and normalize exactly r factors from the transformed data
pca_Y <- svd(Y, nu = r, nv = 0)
Fhat <- sqrt(T) * pca_Y$u[, 1:r, drop = FALSE]

# Identify the lower triangle including the diagonal
lower <- lower.tri(matrix(0, r, r), diag = TRUE)
df <- r * (r + 1) / 2

# Store the nonduplicated factor products for each observation
products <- matrix(0, nrow = T, ncol = df)

for (i in 1:T) {
  factor_product <- tcrossprod(Fhat[i, ])
  products[i, ] <- factor_product[lower]
}

# Compare factor second moments between the two halves
first_sum <- colSums(products[first, , drop = FALSE])
second_sum <- colSums(products[second, , drop = FALSE])

V <- (first_sum - second_sum) / sqrt(T)

# Subtract the corresponding entries of the identity matrix
identity_entries <- diag(r)[lower]
Z <- sweep(products, 2, identity_entries, "-")

# Calculate the lag-zero covariance
Omega <- crossprod(Z) / T

# Add lagged covariances with Bartlett weights
if (bandwidth > 1) {
  for (lag in 1:(bandwidth - 1)) {

    current <- Z[(lag + 1):T, , drop = FALSE]
    previous <- Z[1:(T - lag), , drop = FALSE]

    Gamma <- crossprod(current, previous) / T
    weight <- 1 - lag / bandwidth

    Omega <- Omega + weight * (Gamma + t(Gamma))
  }
}

# Remove small numerical asymmetries
Omega <- (Omega + t(Omega)) / 2
```

```
  # Stop if the covariance cannot be reliably inverted
  if (rcond(Omega) < 1e-10) {
    stop("The estimated long-run covariance is nearly singular")
  }

  # Calculate the Wald statistic and chi-square p-value
  W <- as.numeric(t(V) %*% solve(Omega, V))
  p_value <- pchisq(W, df = df, lower.tail = FALSE)

  # Return the test results
  data.frame(
    r = r,
    bandwidth = bandwidth,
    W = W,
    df = df,
    p_value = p_value
  )
}
```

Calculate the presentation specification We use three factors, bandwidth seven, and transform the 2025 series

```
# Set the significance level
alpha <- 0.05

# Test using three factors and bandwidth six
# A contains 2020 returns and B contains 2025 returns
# The function transforms B
main_result <- common_structure_test(
  A, B,
  r = 3,
  bandwidth = 7
)

# Calculate the chi-square rejection threshold
main_result$critical_value <- qchisq(
  1 - alpha,
  df = main_result$df
)

# Reject the null when the p-value is below alpha
main_result$conclusion <- ifelse(
  main_result$p_value < alpha,
  "Reject H0",
  "Do not reject H0"
)

# Display the results
kable(
  main_result,
  digits = 6,
  caption = "Three factors and bandwidth six with 2025 transformed"
)
```

Table 3: Three factors and bandwidth six with 2025 transformed

r	bandwidth	W	df	p_value	critical_value	conclusion
3	7	17.91753	6	0.006442	12.59159	Reject H0